-- ==
-- OLIST MARKETPLACE ANALYSIS
-- Challenges & Solutions Documentation
-- Purpose: Record of all data quality issues, technical problems,
--   and solutions implemented during warehouse development
-- Author: Ria Singh
-- ==

This document captures real-world data engineering challenges encountered during the Olist project and how they were resolved. These demonstrate problem-solving skills, debugging ability, and understanding of data quality.

==========

BRONZE LAYER CHALLENGES

CHALLENGE 1: Empty String Values in Numeric Columns

Problem:

- products table contained empty strings ('') in numeric columns (product_name_length, product_weight_g, etc.)
- MySQL CAST to INT failed with error: "Incorrect integer value: ''"

Root Cause:

- CSV export from Olist included empty strings instead of NULL for missing values

Solution:

- Changed all numeric columns in bronze.products to VARCHAR data type
- Deferred type conversion to silver layer where CASE WHEN handles empty strings
- Example: CASE WHEN product_weight_g = '' THEN NULL ELSE CAST(...) END

Learning:

- Bronze layer should mirror source exactly, even if data types seem "wrong"
- Type conversion belongs in silver layer where you can handle edge cases

Impact:

- Products table loaded successfully with all 32,951 rows

CHALLENGE 2: Embedded Newlines in Review Text

Problem:

- order_reviews CSV failed to load with error: "Row 77917 was truncated; it contained more data than there were input columns"
- LOAD DATA INFILE could not parse rows where review_comment_message contained literal newline characters (customer pressed Enter while typing)

Root Cause:

- CSV format uses newlines as row delimiters
- When text fields contain embedded newlines, parser treats them as new rows
- Result: One review split into multiple malformed rows

Solution:

- Preprocessed CSV with Python pandas before loading:

```
python

df['review_comment_message'] = df['review_comment_message'].str.replace('\n', ' ')
df.to_csv('olist_order_reviews_cleaned.csv')
```

- Loaded cleaned CSV instead of original

Alternative Considered:

- Using LINES TERMINATED BY '\r\n' vs '\n' — did not solve the issue
- Using ENCLOSED BY '"' with ESCAPED BY — also insufficient

Learning:

- Free-text fields require preprocessing before bulk CSV import
- Pandas handles CSV parsing more robustly than MySQL LOAD DATA INFILE

Impact:

- Successfully loaded 99,224 reviews (10 extra rows from duplicates)

CHALLENGE 3: MySQL secure-file-priv Restriction

Problem:

- LOAD DATA INFILE failed with: "The MySQL server is running with the --secure-file-priv option so it cannot execute this statement"
- MySQL only allows loading files from specific directory

Root Cause:

- Security feature restricts LOAD DATA to prevent arbitrary file access
- Default allowed directory: C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/

Solution:

- Identified allowed directory: SHOW VARIABLES LIKE 'secure_file_priv';
- Moved all CSV files to the approved upload directory
- Updated file paths in LOAD DATA statements

Alternative Considered:

- MySQL Workbench Import Wizard — too slow for 1M+ row geolocation table
- Disabling secure-file-priv in my.ini — unnecessary for local development

Learning:

- Always check MySQL security settings before bulk import operations
- secure-file-priv exists for good reason; work with it, don't disable it

Impact:

- All 9 bronze tables loaded successfully

==========

SILVER LAYER CHALLENGES

CHALLENGE 4: Duplicate review_ids with Different order_ids

Problem:

- Silver validation detected 789 review_ids appearing multiple times
- Same review text and timestamps but different order_ids
- Example: review_id "08528..." appears for 3 different orders with identical content, creation date, and answer timestamp

Root Cause:

- Data quality issue in original Olist dataset
- Likely a system bug where one review was incorrectly duplicated across multiple orders during data export

Solution:

- Used ROW_NUMBER() OVER (PARTITION BY review_id) to keep one row per review
- Selected earliest review by review_creation_date as the "true" record
- Documented as known source data issue, not transformation error

Impact:

- Reduced order_reviews from 99,224 to 98,400 unique reviews
- 91 review_ids had different scores (required AVG in fact_orders)
- 173 review_ids had identical scores (no effect on aggregation)

Learning:

- Real-world data is messy; source systems have bugs
- Document data quality issues discovered, don't hide them
- Deduplication strategy should be deterministic (earliest date, not random)

CHALLENGE 5: Geolocation Deduplication Strategy

Problem:

- Geolocation table had ~1 million rows for only ~20k unique zip codes
- Same zip code appeared with different city spellings (São Paulo vs Sao Paulo)
- Also appeared with genuinely different cities (data entry errors)

Root Cause:

- Crowdsourced geolocation data with no validation
- Multiple sellers/customers entering location data inconsistently

Solution:

- Implemented mode-based deduplication:

```sql
```

```
ROW_NUMBER() OVER (
  PARTITION BY geolocation_zip_code_prefix
  ORDER BY COUNT(*) OVER (PARTITION BY zip, city, state) DESC
)
```

- Kept the most frequently occurring city/state per zip code
- Assumption: Majority spelling is likely correct

Alternative Considered:

- Keep all variations — breaks star schema (dimension needs one row per entity)
- Use external geocoding API — overkill for portfolio project

Learning:

- When deduplicating with conflicts, choose a tie-breaker (frequency, recency)
- Document the assumption (majority = truth) so others understand the logic

Impact:

- Reduced geolocation from 1,000,163 rows to 19,015 unique zip codes

CHALLENGE 6: Window Function Nesting Not Supported

Problem:

- Attempted to use COUNT(*) window function inside ORDER BY of another window function for geolocation deduplication
- MySQL error: "You cannot nest a window function in the specification of window '<unnamed window>'"

Root Cause:

- MySQL does not allow window functions to reference other window functions in the same query level

Solution:

- Split into multi-stage CTE approach:
  1. First CTE: Calculate occurrence count for each zip+city+state combo
  2. Second CTE: Apply ROW_NUMBER() using the pre-calculated count
  3. Final SELECT: Filter WHERE rn = 1

Learning:

- Complex window function logic requires breaking into CTEs
- CTEs improve readability even when not strictly required

Impact:

- Query executed successfully, proper deduplication achieved

==========

GOLD LAYER CHALLENGES

CHALLENGE 7: Cartesian Product in fact_orders Aggregation

Problem:

- Payment values in fact_orders were inflated by 8-12x

- Example: Order with 8 items and ₹13,664 payment showed as ₹109,312
- Validation check found 10,326 orders (10.5%) with payment > ₹100 different from order revenue

Root Cause:

- Original query structure:

```
sql

FROM orders
LEFT JOIN order_items (1-to-many)
LEFT JOIN order_payments (1-to-1 with orders)
GROUP BY order_id
SUM(payment_value)
```

- Because order_items is joined first (creating 8 rows for order with 8 items), the subsequent payment join duplicates payment_value 8 times
- SUM then adds up all 8 copies: 13,664 × 8 = 109,312

Debugging Process:

1. Noticed large payment vs revenue discrepancies in validation
2. Sampled problem orders and checked raw payment_value in order_payments
3. Compared row count in intermediate join vs final fact table
4. Discovered row_count matched number of items (cartesian product confirmed)

Solution:

- Refactored to use CTEs that pre-aggregate before joining:

```
sql

WITH order_aggregates AS (
  SELECT order_id, SUM(price) as total_price FROM order_items GROUP BY order_id
),
payment_aggregates AS (
  SELECT order_id, SUM(payment_value) as total_payment FROM order_payments GROUP BY order_id
)
SELECT * FROM orders
LEFT JOIN order_aggregates ...
LEFT JOIN payment_aggregates ...
```

- Each CTE groups to order_id level independently
- No cartesian product when joining pre-aggregated CTEs

Impact:

- Reduced large discrepancies from 10,326 orders (10.5%) to 1,110 orders (1.1%)
- Remaining discrepancies are legitimate (vouchers, refunds)
- Average difference dropped from ₹72.20 to ₹22.85

Learning:

- Always aggregate related entities separately before joining to fact grain
- CTEs prevent subtle bugs in complex multi-table aggregations
- Validation queries catch these issues — always validate aggregated metrics

CHALLENGE 8: MySQL Connection Timeouts During UPDATE

Problem:

- Attempted to add customer_unique_id to fact_orders via UPDATE + JOIN
- Query repeatedly lost connection to MySQL server during execution
- Even DROP TABLE commands were timing out and waiting for locks

Root Cause:

- Large UPDATE with JOIN on 99,441 rows overwhelmed MySQL connection
- UPDATE held table lock, blocking all subsequent operations
- Multiple stuck processes accumulated, each waiting for the lock

Debugging Process:

1. Ran SHOW PROCESSLIST to identify stuck queries
2. Found UPDATE running for 558 seconds holding lock on fact_orders
3. Multiple DROP TABLE commands waiting for "table metadata lock"

Solution:

- Killed stuck UPDATE process: KILL <process_id>
- Abandoned UPDATE approach entirely
- Instead, recreated fact_orders with customer_unique_id in original CREATE TABLE statement by adding JOIN to customers table

Alternative Considered:

- Batched UPDATE in chunks of 25k rows — still too slow
- CREATE INDEX before UPDATE — helps but doesn't solve timeout issue

Learning:

- For large transformations, recreating table is often faster than UPDATE
- "Fix it at creation time" beats "fix it after the fact"
- SHOW PROCESSLIST is essential for diagnosing lock issues

Impact:

- Table recreated in 20 seconds vs. hung UPDATE that never completed
- Proper foreign key relationship established (customer_unique_id → dim_customers)

CHALLENGE 9: MySQL Recursion Depth Limit for dim_date

Problem:

- dim_date generation using recursive CTE failed with: "Recursive query aborted after 1001 iterations"
- Needed to generate 1,096 dates (2016-2018) but MySQL default limit is 1000

Root Cause:

- MySQL safety feature prevents infinite recursion
- Default @@cte_max_recursion_depth = 1000

Solution:

- Increased session recursion limit before CREATE TABLE:

```sql
SET SESSION cte_max_recursion_depth = 2000;
```

- Query executed successfully, generated all 1,096 dates

Alternative Considered:

- Generate dates in Python, export CSV, load as table — unnecessary complexity
- Split into multiple CTEs for different year ranges — overly complex

Learning:

- Know MySQL's safety limits (recursion, execution time, memory)
- Temporarily increasing limits for legitimate use cases is acceptable
- Always reset limits after use (SET SESSION is temporary anyway)

Impact:

- dim_date fully populated with 1,096 days from 2016-01-01 to 2018-12-31

==========

VALIDATION & DATA QUALITY FINDINGS

FINDING 1: Low Customer Repeat Purchase Rate

Observation:

- 99,441 orders from 96,096 unique customers
- Average orders per customer: 1.03
- Only 3,345 customers (3.5%) made multiple purchases

Implication:

- Olist marketplace has low customer retention
- Business opportunity: Improve repeat purchase incentives
- Analysis focus should include customer acquisition cost vs. lifetime value

Action Taken:

- Documented in dim_customers validation
- Will explore in RFM analysis (Recency, Frequency, Monetary segmentation)

FINDING 2: 13% of Orders Have No Reviews

Observation:

- 98,135 orders with reviews (98.7%)
- 1,306 orders without reviews (1.3%)

Possible Reasons:

- Cancelled orders (review not possible)
- Customer chose not to review
- Review was deleted

Implication:

- Review score metrics should handle NULL appropriately
- Average review score (4.09/5) represents only reviewed orders

Action Taken:

- fact_orders keeps review_score as NULL (not 0) for orders without reviews
- Validation queries use COUNT(CASE WHEN review_score IS NOT NULL)

FINDING 3: Cancelled Orders Represented as ₹0 Revenue

Observation:

- Minimum order value: ₹0.00
- These are orders with status = CANCELLED or UNAVAILABLE

Decision:

- Keep cancelled orders in fact_orders (not filtered out)
- Enables cancellation rate analysis
- Shows complete order funnel, not just successful orders

Business Metrics Enabled:

- Cancellation rate by state/category
- Late delivery → cancellation correlation
- Order status distribution

FINDING 4: Late Delivery Rate of 6.57%

Observation:

- 6,535 orders delivered late (positive delivery_delay_days)
- 89,941 orders delivered on-time or early
- Late delivery rate: 6.57%

Implication:

- Olist has relatively good delivery performance
- Late deliveries likely correlate with lower review scores
- Geographic analysis needed (are certain states consistently late?)

Action Taken:

- Created delivery_delay_days metric in fact_orders
- Negative = early (good), Positive = late (bad), Zero = perfect
- Enables analysis of delivery performance impact on customer satisfaction

==========

TECHNICAL BEST PRACTICES DEMONSTRATED

1. Medallion Architecture (Bronze → Silver → Gold)
    - Separates raw ingestion from cleaning from analytics
    - Each layer serves different purpose and audience
    - Enables data lineage tracking and debugging
2. Defensive SQL (COALESCE, NULLIF, CASE WHEN)
    - Handles edge cases (empty strings, NULL values)

- Prevents errors from unexpected data conditions
- Makes transformations explicit and auditable

3. CTE-based Aggregation
    - Prevents cartesian products in complex multi-table joins
    - Improves query readability (each CTE has single clear purpose)
    - Easier to debug (can query CTEs independently)
4. Comprehensive Validation
    - Row count checks at each layer
    - Referential integrity validation (no orphan records)
    - Business logic validation (no negative prices, valid date ranges)
    - Duplicate detection and resolution
5. Documentation as Code
    - Every design decision explained in SQL comments
    - Challenges and solutions recorded
    - Future maintainer can understand the "why" not just the "what"

==========

INTERVIEW TALKING POINTS

When asked "What challenges did you face?", focus on these narratives:

1. DATA QUALITY CHALLENGE (reviews with embedded newlines) → Demonstrates data profiling, problem diagnosis, Python preprocessing
2. PERFORMANCE ISSUE (cartesian product in fact_orders) → Shows debugging complex SQL, understanding join mechanics, refactoring skills
3. DESIGN DECISION (geolocation mode-based deduplication) → Illustrates trade-off analysis, assumption documentation, business judgment

These show you can:

- Diagnose root causes (not just symptoms)
- Evaluate multiple solutions
- Make informed engineering decisions
- Document for future maintainers

==========

=